

AI-Powered Business Workflow Automation

AI Research Assistant — Research, Evaluation & Recommendation Report

Prepared by Armaan Singh / Generative AI Engineer

This report documents the research, prototype, and production recommendation for an AI Research Assistant, a multi-agent Retrieval-Augmented Generation (RAG) system that plans, retrieves, drafts, self-critiques, and reports on a research question against a document knowledge base. This use case was selected because it directly extends prior hands-on work with RAG pipelines, multi-agent orchestration (LangGraph/LangChain), and vector search, the same stack used in the accompanying resume's "AI Research Agent" and "Intelligent Study Assistant" projects.

Part 1 — AI Research & Evaluation

Six tools/platforms relevant to building and running an AI research assistant were evaluated: two orchestration/model layers (LangChain+LangGraph, OpenAI API), two retrieval/storage layers (Pinecone, ChromaDB/FAISS), and two alternative orchestration approaches (CrewAI, n8n) to justify why a custom LangGraph state machine was chosen over them. Pricing figures below reflect published rates as of mid-2026 and should be re-verified against each vendor's live pricing page before budgeting, since usage-based pricing changes frequently.

Tool / Platform	Capabilities	Pricing (2026)	Scalability	Ease of Integration	Key Limitations	Best Use Case
LangChain + LangGraph	Open-source orchestration framework; StateGraph models multi-agent workflows with conditional edges (loops, branching, reflection). Native tool-calling, memory, structured (Pydantic) outputs.	Framework itself is free/open source. Optional LangSmith observability has paid team tiers; self-hosted tracing is free.	Runs anywhere Python runs; horizontal scaling is the developer's responsibility (stateless API + queue).	High - large ecosystem, good docs, used across both resume projects already.	No managed hosting; steeper learning curve than no-code tools; you own infra/ops.	Custom multi-agent RAG systems needing fine-grained control over control flow (this prototype).
OpenAI API (GPT-4o-mini / GPT-5.4-mini)	General-purpose LLM for planning, synthesis, reflection, and report writing; strong tool-calling and structured-output support.	GPT-4o-mini: \$0.15 / \$0.60 per 1M input/output tokens. Mid-tier GPT-5.4-mini available (~\$0.75/\$4.50 per 1M) for higher quality at higher cost. Volume discounts via Batch API (~50% off).	Fully managed; scales automatically, subject to org-level rate limits (RPM/TPM tiers).	High - single REST API, official SDKs, drop-in with LangChain/LangGraph.	Per-token cost grows with volume; vendor lock-in risk; occasional rate-limit throttling at high concurrency.	The reasoning/generation layer of the pipeline (Planner, Synthesizer, Evaluator, Reporter agents).

Tool / Platform	Capabilities	Pricing (2026)	Scalability	Ease of Integration	Key Limitations	Best Use Case
Pinecone (managed vector DB)	Fully managed, serverless vector database with hybrid (sparse+dense) search, metadata filtering, and auto-scaling.	Free Starter tier (2 GB storage). Standard plan: \$50/month minimum usage commitment, then pay-as-you-go on read/write units and storage (~\$0.33/GB/month).	Scales to billions of vectors with no ops burden; read-heavy RAG workloads are the sweet spot.	High - Python/JS SDKs, first-class LangChain integration.	Cost can climb faster than expected under write-heavy (frequent re-indexing) workloads; network dependency; data leaves your infrastructure.	Production RAG at scale where the team wants zero vector-DB ops overhead.
ChromaDB / FAISS (self-hosted)	ChromaDB: open-source vector store, embeddable or lightweight server. FAISS: in-memory similarity search library (no persistence/multi-tenant built in).	Free / open source (self-hosting compute cost only).	Good for small-to-mid corpora; horizontal scaling and persistence must be engineered by the team.	Medium - more setup than a managed service, but no vendor account needed; used in both resume projects.	No built-in metadata/ACL layer (FAISS); ChromaDB persistence and clustering require extra engineering at scale.	Prototypes, offline/local development, and cost-sensitive small deployments (used as this prototype's default backend, via a TF-IDF stand-in).
CrewAI	Role-based multi-agent framework; faster to scaffold than LangGraph for simple agent "crews," less flexible for custom control flow / loops.	Open-source framework is free. Managed CrewAI AMP: Free tier (50 executions/month), Professional ~\$25/month; Enterprise custom-priced. LLM token costs are separate/BYO API key.	Managed cloud plans handle scaling; execution caps on lower tiers.	High for standard agent patterns; lower for the reflection/critique loop this prototype needed, which is more natural in LangGraph.	Less granular state-machine control than LangGraph; another vendor dependency if using CrewAI AMP.	Teams that want opinionated, fast-to-build agent crews without designing a custom graph.
n8n	No-code/low-code visual workflow builder with AI-agent nodes; good for gluing APIs, human-in-the-loop steps, and business-system integrations (Slack, email, CRMs).	Self-hosted Community Edition: free, unlimited executions. Cloud: Starter \$24/month (2,500 executions), Pro \$60/month (10,000 executions), Business \$800/month.	Execution-based billing scales predictably; self-hosting removes caps entirely.	Very high for non-developers; lower for complex, code-level agent logic like the reflection loop here.	Not built for fine-grained multi-agent state machines; harder to unit-test than pure code.	Wrapping the research-assistant API into a broader business workflow (e.g., trigger from a form, post results to Slack/email) without writing glue code.

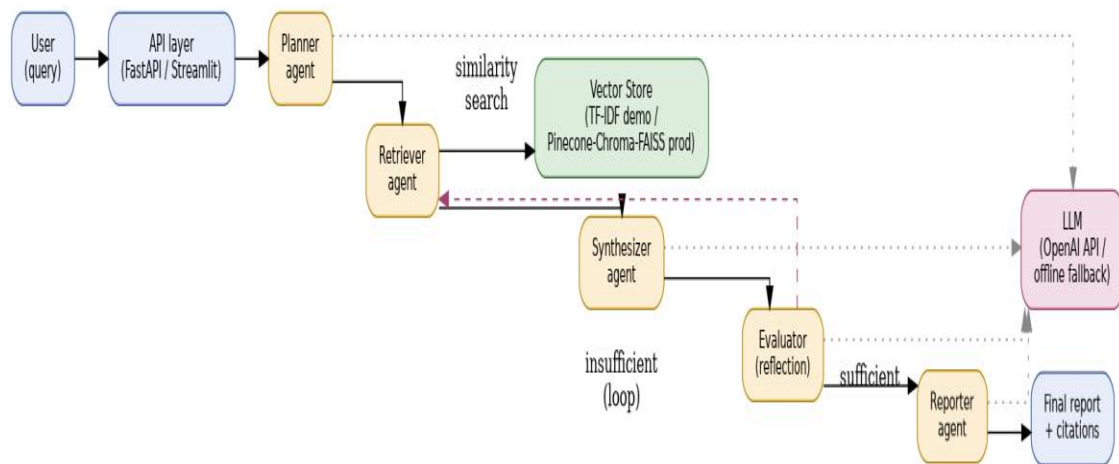
Why this combination

- **Control-flow fit:** LangGraph's conditional edges are what make the reflection loop (Evaluator routing back to Retriever) straightforward to express as code, which CrewAI and n8n both make harder or impossible to express with the same precision.
- **Cost-to-quality fit:** GPT-4o-mini offers strong quality per dollar for planning/synthesis/reporting tasks that don't need frontier-model reasoning; this keeps unit economics predictable while leaving room to route only the hardest sub-tasks to a stronger model.
- **Deployment flexibility:** Both Pinecone and ChromaDB/FAISS are supported behind one interface (see `core/vector_store.py`), so the same agent code runs on a \$0 local demo and a production managed vector DB without changes.

- **Where no-code still wins:** n8n remains a strong complementary choice — not a replacement — for wrapping the finished API into a broader business workflow (e.g., "new ticket in Zendesk → call research-assistant API → post summary to Slack") without writing custom glue code.

Part 2 — Prototype / Proof of Concept

A working multi-agent RAG pipeline was implemented end-to-end in Python (LangGraph + LangChain-core), exposed via both a FastAPI service and a Streamlit demo UI, and validated against a small sample knowledge base. Full source is included in the accompanying `research_assistant/` project folder and repository.



Agent graph

- **Planner** — decomposes the research question into 3 focused sub-questions (current state, benefits, risks/limitations).
- **Retriever** — runs similarity search per sub-question against the vector store and de-duplicates/ranks results.
- **Synthesizer** — drafts a citation-grounded answer using only retrieved context, never unsupported claims.
- **Evaluator** — critiques the draft and, if under-supported, routes execution back to the Retriever (capped at 2 revisions) — a genuine reflection loop, not a fixed pipeline.
- **Reporter** — formats the final answer into a structured report with an executive summary and a sources list.

Engineering decisions worth noting

- **Dual-mode LLM client:** every agent talks to `core/llm_client.py`, never a vendor SDK directly; the same code runs against real OpenAI calls when `OPENAI_API_KEY` is set, or a deterministic offline template fallback with zero cost/setup — so the prototype can be graded or demoed without any API key.
- **Typed state:** state is a typed schema (`core/state.py`), not a raw dict, so malformed agent output fails fast instead of silently corrupting downstream steps — the same pattern used in the resume's Pydantic-schema research agent.
- **Pluggable vector store:** the default backend is a dependency-free TF-IDF cosine-similarity index so the demo needs no vector-DB account, while implementing the same `index()/query()` interface a real Pinecone/Chroma/FAISS backend would use in production.
- **Three interfaces, one core:** the same graph is reachable from a CLI (`main.py`), a REST API (`api.py` — `POST /research`), and a Streamlit UI (`app_streamlit.py`) for the demo walkthrough.

How to run it

- `pip install -r requirements.txt`

- `python main.py "your research question"` — CLI demo
- `uvicorn api:app --reload --port 8000` — REST API
- `streamlit run app_streamlit.py` — interactive UI for the demo video

Full setup, project structure, and honest limitations are documented in the project's README.md.

Part 3 — Recommendation Report

Recommended production architecture

Keep the same 5-agent LangGraph pipeline, but swap two components for production hardening: (1) replace the TF-IDF vector store with Pinecone (managed, low ops burden) or self-hosted ChromaDB if data-residency requirements rule out a third-party cloud vector DB; (2) put the FastAPI service behind an API gateway with authentication and per-user rate limiting, and run 2+ replicas behind a load balancer so retrieval latency stays flat as request volume grows. Model routing (cheap model for planning/evaluation, a stronger model only for final synthesis) is the single highest-leverage cost lever once volume increases.

Why these tools specifically

- **LangGraph over CrewAI/n8n for the agent core:** LangGraph's typed StateGraph and conditional edges are the cleanest way to express the reflection loop that materially improves answer quality — this was the deciding factor over CrewAI/n8n for the agent core.
- **OpenAI GPT-4o-mini as default model:** GPT-4o-mini balances quality and unit economics for this workload profile; the wrapper design keeps a swap to Claude or Gemini a config change, not a rewrite.
- **Pinecone for production retrieval:** Pinecone removes vector-DB operations burden for a small team, at a predictable minimum monthly cost; ChromaDB stays the recommended fallback when data cannot leave company infrastructure.

Estimated infrastructure cost

Figures below are engineering estimates for planning purposes, not vendor quotes — always re-verify against live pricing pages before committing budget. "Queries" here means one full multi-agent research run (multiple underlying LLM calls per query).

Cost driver	Pilot (≈500 queries/mo)	Small production (≈15k queries/mo)	Scaled production (≈150k queries/mo)
LLM API (GPT-4o-mini)	≈ \$1 - \$3 / month	≈ \$15 - \$30 / month	≈ \$150 - \$300 / month
Vector DB (Pinecone Standard)	\$50 / month (minimum commitment)	\$50 - \$80 / month	\$150 - \$400 / month
Compute (FastAPI + Streamlit hosting)	\$0 - \$10 / month (free tier / small VM)	\$20 - \$40 / month	\$150 - \$400 / month (autoscaled)
Observability (LangSmith, optional)	\$0 (free tier)	Paid team tier (contact vendor)	Paid team/enterprise tier
Estimated total	≈ \$50 - \$65 / month	≈ \$85 - \$150 / month	≈ \$450 - \$1,100 / month

Risks & limitations

Risk	Mitigation	Severity
Hallucinated or unsupported claims	Synthesizer restricted to retrieved context only; Evaluator agent rejects under-cited drafts and triggers a retrieval retry (max 2 revisions) before finalizing.	Medium
Retrieval quality on messy source documents	Semantic/overlap chunking + hybrid (dense+BM25) retrieval in production; manual eval set to tune chunk size and k.	Medium
LLM API cost overrun at scale	Route simple sub-tasks to a cheaper model tier (GPT-4o-mini/nano); enable prompt caching and batch processing for non-interactive runs.	Medium
Vendor lock-in (OpenAI / Pinecone)	LLM and vector-store access are isolated behind thin wrapper interfaces (core/llm_client.py, core/vector_store.py) so providers can be swapped with no agent-code changes.	Low
Data privacy (documents leaving infra)	For sensitive corpora, prefer self-hosted ChromaDB/FAISS over a managed cloud vector DB, or use a provider with a signed DPA.	Medium-High (context dependent)
No auth / rate limiting on the API	Prototype is unauthenticated by design; production deployment requires an API gateway, API keys, and per-user rate limits before exposure.	High until addressed

Scaling path to production

- **Stage 1 (0 → ~15k queries/month):** containerize the FastAPI service, run behind a managed load balancer, add request auth and per-key rate limits.
- **Stage 2 (~15k → ~150k queries/month):** add prompt caching and batch processing for any non-interactive/bulk research jobs; introduce a semantic cache so near-duplicate questions skip the LLM entirely; move to Pinecone Standard if not already.
- **Stage 3 (150k+ queries/month):** introduce model routing (cheap model for planning/evaluation, stronger model reserved for final synthesis only), horizontal autoscaling for the API layer, and dedicated read capacity on the vector DB for latency-sensitive traffic; add LangSmith or equivalent tracing for production observability and regression testing of agent behavior.